

SECURITY & ARCHITECTURE WHITEPAPER

KIẾN TRÚC PHIÊN VÀ CƠ CHẾ XOAY VÒNG TOKEN TRÊN REDIS

Chuyên sâu về Token Rotation, Grace Period Đa Tab và Phát Hiện Xâm Nhập (Reuse Detection)

HỆ THỐNG

Tomato Cinema

CHUYÊN ĐỀ

Security & Session Mgmt

TIÊU CHUẨN

OAuth 2.0 BCP (RFC 6749)

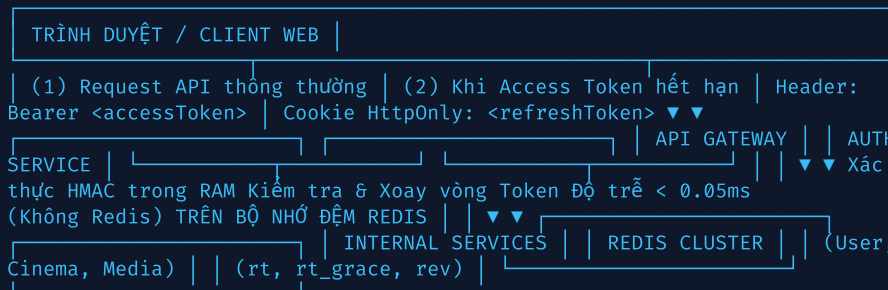
PHIÊN BẢN

v1.2.0 (Deep-Dive Edition)

1. Tổng quan Kiến trúc Xác thực Hybrid (Stateless vs. Stateful)

Hệ thống **Tomato Cinema** áp dụng mô hình xác thực lai (Hybrid Token Authentication) kết hợp ưu điểm của cả hai thế giới:

- Access Token là Stateless:** Hoàn toàn không lưu trữ trong Redis hay Cơ sở dữ liệu, giúp thẩm định chữ ký với độ trễ cực thấp.
- Refresh Token là Stateful:** Được giám sát, lưu vết và kiểm soát vòng đời chặt chẽ trên bộ nhớ đệm phân tán Redis.



2. Tại sao Access Token tuyệt đối KHÔNG lưu trong Redis?

Một thách thức kinh điển của các kỹ sư backend: "Tại sao khi kiểm tra Redis lại không thấy bất kỳ Access Token nào?". Đây là quyết định kiến trúc có tính toán kỹ lưỡng:

- Hiệu năng cực đại (Zero-I/O Verification):** Nếu lưu Access Token vào Redis, mỗi request từ client (hàng chục nghìn req/giây) đều phải tốn một round-trip mạng tới Redis. Tại Tomato Cinema, Access Token được ký bằng HMAC SHA-256 qua gói `@tomatocinema/passport`. API Gateway chỉ cần thẩm định chữ ký toán học ngay trên RAM Node.js với độ trễ < **0.05ms**.
- Thời gian sống ngắn (Short-lived):** Access Token chỉ có thời hạn 1 giờ (3,600s). Nếu lỡ bị lộ, thời gian hacker có thể lợi dụng là rất ngắn.
- Bảo vệ dung lượng RAM Cache:** Giúp Redis không bị phình to bởi hàng triệu token ngắn hạn, dành trọn vẹn RAM cho các tác vụ quan trọng như Session, Seat Locking (khóa ghế xem phim) và Cache danh mục phim.

3. Cơ chế 3 Nhóm Khóa Refresh Token trong Redis

Refresh Token có thời hạn dài (24 giờ đến 30 ngày) nên bắt buộc phải được quản lý trạng thái (Stateful) để hỗ trợ thu hồi tức thì và phát hiện xâm nhập. Nhằm bảo mật tối đa, hệ thống băm SHA-256 lấy 32 ký tự đầu làm dấu vân tay (`{fingerprint}`):

```
const fingerprint = createHash('sha256').update(refreshToken).digest('hex').slice(0, 32);
```

💡 Lợi ích: Chuỗi token thô không bao giờ nằm trong Redis. Ngay cả khi hacker dump được RAM Redis, chúng cũng không thể suy ngược ra Refresh Token gốc của người dùng.

Khóa Redis	Thời gian (TTL)	Mục đích kỹ thuật	Dung lượng & Dữ liệu
<code>rt:{userId}:{fp}</code>	24 giờ / 30 ngày	Active Session: Phiên Refresh Token hợp lệ duy nhất đang hoạt động.	<code>{"userId":" ... ", ... }</code> (~181 Bytes)
<code>rt_grace:{userId}:{fp}</code>	30 giây	Grace Period (Cửa sổ Ân hạn): Lưu cập token mới xử lý Race Condition khi mở nhiều tab.	<code>{"accessToken":" ... ", ... }</code> (~491 Bytes)
<code>rt_revoked:{userId}:{fp}</code>	24 giờ	Revocation Tracking: Lưu vết token đã xoay vòng để bắt tấn công tái sử dụng (Reuse Detection).	Chuỗi <code>"1"</code> (~109 Bytes)

★ 3.1. Mô hình Ví von Đời thường: "Hệ Thống Thẻ Từ Khách Sạn Thông Minh"

Để trực quan hóa cơ chế vận hành của 3 chiếc chìa khóa trên, hãy tưởng tượng hệ thống bảo mật này như **Hệ thống quản lý thẻ từ thông minh tại một khách sạn 5 sao**:

**1. Khóa `rt` = "Thẻ Từ Phòng Khách Sạn VIP Đang Hiệu Lực"**

ACTIVE KEY

Đây là chiếc thẻ phòng chính chủ duy nhất bạn đang giữ trong tay. Thẻ này cho phép bạn mở cửa vào phòng xem phim và dùng để xuất trình cho nhân viên lễ tân khi bạn muốn xin đổi thẻ mới. Mỗi phòng/thiết bị tại một thời điểm chỉ có **duy nhất 1 chiếc thẻ VIP** có giá trị.

**2. Khóa `rt_grace` = "Chiếc Thẻ Thay Thế Tạm Thời Trong 30 Giây"**

GRACE PERIOD

Khi bạn vừa đổi thẻ mới tại quầy, lễ tân chu đáo dặn: *"Trong vòng 30 giây tới, nếu bạn hoặc người thân của bạn lỡ quẹt chiếc thẻ cũ vào cửa phòng, máy quét vẫn mở cửa và tự động nhả ra chiếc thẻ mới cho bạn"*. Nhờ cơ chế này, nếu bạn mở cùng lúc 3 hay 5 tab trình duyệt, các tab phụ gửi yêu cầu trễ vài giây vẫn nhận được token mới ngon lành mà không bị lễ tân đuổi ra ngoài!

**3. Khóa `rt_revoked` = "Sổ Đen & Bẫy Chuông Báo Động Bất Kê Trộm"**

REUSE TRAP

Sau khi hết 30 giây ân hạn, chiếc thẻ cũ bị vô hiệu hóa hoàn toàn và số serial của nó được ghi vào **Sổ Đen (`rt_revoked`)** trong 24 giờ. Nếu sau đó có kẻ gian (Hacker) nhặt được thẻ cũ đem quẹt vào cửa phòng → **CHUÔNG BÁO ĐỘNG REO LÊN!** Đội bảo vệ lập tức khóa toàn bộ cửa phòng, hủy luôn cả chiếc thẻ mới của bạn và yêu cầu chủ nhân phòng trình diện căn cước để mở lại. Kẻ trộm bị chặn đứng 100%.

4. Phân tích Dữ liệu Thực tế trên Redis (Deep-Dive Case Study)

Dưới đây là dữ liệu thực tế thu thập từ giao diện Redis Commander của phiên người dùng `bff2dc4d-d60e-444b-9e0a-54b7649b7838`:

```
1. Nhóm Active Refresh Token (rt):
  └─ rt:bff2dc4d-d60e-444b-9e0a-54b7649b7838:ec524309067f0a92ff98a91dc55a067f
     TTL: 23 h | Dung lượng: 181 B (Token mới nhất đang hoạt động)

2. Nhóm Cửa sổ Ân hạn (rt_grace):
  └─ rt_grace: ... :0be92051a829f1381ea0f8b9580a4a9e (TTL: 25 s | 491 B)
     rt_grace: ... :62592a2ad1cb8f07722522b326606990 (TTL: -2 s | Vừa hết hạn)
     rt_grace: ... :a8793ff75be4a208fe64a70e14319797 (TTL: 11 s | 491 B)

3. Nhóm Lịch sử Thu hồi (rt_revoked):
  └─ rt_revoked: ... :0be92051a829f1381ea0f8b9580a4a9e (TTL: 23 h | 109 B | Value: "1")
     rt_revoked: ... :62592a2ad1cb8f07722522b326606990 (TTL: 23 h | 109 B | Value: "1")
     rt_revoked: ... :a8793ff75be4a208fe64a70e14319797 (TTL: 23 h | 109 B | Value: "1")
```

Hiện tượng gì vừa diễn ra trong hệ thống?

Người dùng (hoặc trình duyệt có nhiều tab) vừa thực hiện **3 lần xoay vòng token liên tiếp** trong vòng chưa đầy 30 giây qua:

- Lần 1:** Sử dụng token `62592a2a ...` → Sinh ra token mới. Token cũ chuyển vào `rt_grace` (nay đã quá 30s nên TTL hiển thị -2s) và lưu vết vào `rt_revoked`.
- Lần 2:** Sử dụng token `a8793ff7 ...` → Sinh ra token mới. Token cũ vào `rt_grace` (còn 11s) và ghi vết vào `rt_revoked`.
- Lần 3:** Sử dụng token `0be92051 ...` → Cấp ra token `ec524309 ...` (đang Active trong `rt`, TTL 23h). Token `0be92051 ...` vào `rt_grace` (còn 25s) và ghi vết vào `rt_revoked`.

? Tại sao `rt_grace` có dung lượng 491 Bytes?

Vì `rt_grace` lưu trọn bộ cả cặp `{ accessToken, refreshToken }` mới dạng chuỗi JSON nén, sẵn sàng phản hồi ngay lập tức cho bất kỳ tab nào gửi lại token cũ trong vòng 30s.

? Tại sao `rt_revoked` chỉ có 109 Bytes?

Vì `rt_revoked` chỉ lưu giá trị chuỗi `"1"` làm cờ đánh dấu (Tombstone Flag). Khi có hàng triệu người dùng, điều này giúp tiết kiệm tối đa bộ nhớ RAM của Redis.

★ 4.1. Bảng Trục Quan 3 Tình Huống Kịch Bản Đời Thường

Tình huống xảy ra	Chuyện gì diễn ra bên trong Redis?	Trải nghiệm & Kết quả
1. Dùng bình thường (Mở 1 tab duy nhất)	Khóa <code>rt</code> cũ bị xóa, khóa <code>rt</code> mới được tạo. Bản sao token mới lưu vào <code>rt_grace</code> (30s) và đánh dấu <code>rt_revoked</code> (24h).	THÀNH CÔNG Người dùng tiếp tục xem phim liền mạch, không hề nhận thấy gián đoạn.
2. Mở nhiều tab cùng lúc (Race Condition)	Tab 1 xoay token xong. Các tab 2, 3, 4 gửi token cũ tới sau vài giây → Trúng ngay <code>rt_grace</code> → Redis trả ngay cặp token mới lưu sẵn.	ÂN HẠN MUỘT MÀ Cả 4 tab đều nhận token mới, không có tab nào bị lỗi hoặc vắng ra màn hình đăng nhập.
3. Kẻ gian tấn công (Hacker dùng token cũ)	Hacker dùng token cũ sau 30 giây (<code>rt_grace</code> đã hết hạn). Request va phải <code>rt_revoked</code> → Phát hiện Reuse Attack!	BẢO ĐỘNG ĐỎ Hệ thống xóa sạch toàn bộ session của tài khoản trên Redis. Hacker bị đá văng tức thì!

5. Thuật toán Xoay Vòng và Phòng Vệ Chuyên Sâu

Dưới đây là mã nguồn lõi của phương thức `rotate(refreshToken)` trong `TokenService` của `auth-service`:

```
public async rotate(refreshToken: string): Promise<{ accessToken: string; refreshToken: string }> {
    // 1. Thẩm định chữ ký HMAC SHA-256 của chuỗi Token
    const result = this.passportService.verify(refreshToken, 'refresh');
    if (!result.valid) {
        throw new RpcException({ code: RpcStatus.UNAUTHENTICATED, details: result.reason });
    }

    const { userId } = result;
    const fingerprint = this.computeFingerprint(refreshToken);

    // 2. BƯỚC CỨU TINH ĐA TAB (Grace Period 30s)
    const graceKey = `rt_grace:${userId}:${fingerprint}`;
    const graceSession = await this.redisService.get(graceKey);
    if (graceSession) {
        return JSON.parse(graceSession); // Trả ngay cặp token mới sinh cho các tab khác
    }

    // 3. BƯỚC BẮT KẼ TRỘM (Token Reuse Detection)
    const revokedKey = `rt_revoked:${userId}:${fingerprint}`;
    const isReused = await this.redisService.get(revokedKey);
    if (isReused) {
        // Tín hiệu báo động đỏ: Có kẻ đang dùng lại token cũ ngoài 30s!
        await this.revokeAllRefreshTokens(userId);
        throw new RpcException({
            code: RpcStatus.UNAUTHENTICATED,
            details: 'Phát hiện tái sử dụng token bất thường. Toàn bộ phiên đã bị thu hồi!'
        });
    }

    // 4. Kiểm tra Active Session trong Redis
    const redisKey = `rt:${userId}:${fingerprint}`;
    const session = await this.redisService.get(redisKey);
    if (!session) {
        throw new RpcException({ code: RpcStatus.UNAUTHENTICATED, details: 'Phiên không hợp lệ hoặc đã hết hạn' });
    }

    // 5. Cấp cặp Token mới
    const newTokens = await this.generate(userId);

    // 6. Cập nhật Redis nguyên tử bằng Pipeline (Atomic Execution)
    const pipeline = this.redisService.pipeline();
    pipeline.del(redisKey); // Xóa active cũ
    pipeline.set(graceKey, JSON.stringify(newTokens), 'EX', 30); // Ấn hạn 30s
    pipeline.set(revokedKey, '1', 'EX', 86400); // Lưu vết 24h
    await pipeline.exec();

    return newTokens;
}
```

6. Hai Bài Toán Kinh Điển Được Giải Quyết Triệt Đê

6.1. Bài toán Race Condition khi Mở Nhiều Tab Trình Duyệt

- Tình huống:** Người dùng mở đồng thời 5 tab xem phim trên Tomato Cinema. Sau 1 giờ, Access Token ở cả 5 tab cùng hết hạn. Trình duyệt gửi đồng thời 5 request `POST /auth/refresh`.
- Cách xử lý:** Tab đầu tiên đến Auth Service xoay vòng token thành công, lưu bản sao vào `rt_grace` trong 30s. 4 Tab tiếp theo đến sau vài chục mili-giây, cầm token cũ hỏi hệ thống → Khớp ngay trong `rt_grace` → Cả 4 tab đều nhận được token mới.
- Kết quả:** Không có bất kỳ tab nào bị gián đoạn hay bị văng ra màn hình đăng nhập (Zero Disruption).

6.2. Bài toán Chống Kẻ Gian Đánh Cắp Token (Replay Attack)

- Tình huống:** Kẻ tấn công bằng cách nào đó đã trộm được chuỗi Refresh Token cũ của người dùng.
- Cách xử lý:** Token cũ sau 30 giây đã hết hạn trong `rt_grace`. Khi hacker cố dùng token này để refresh lấy Access Token mới → Vướng ngay vào chiếc bẫy `rt_revoked`.
- Phản ứng của hệ thống:** Ngay lập tức gọi hàm `revokeAllRefreshTokens(userId)` quét sạch tất cả các session của userId đó trên Redis bằng lệnh `SCAN`.
- Kết quả:** Cả hacker lẫn phiên đăng nhập hiện tại đều bị đá văng tức thì. Tài khoản người dùng được bảo vệ 100% trước nguy cơ bị chiếm quyền âm thầm.

7. Bảng Ma Trận So Sánh Tổng Kết

Tiêu chí so sánh	Access Token	Refresh Token
Bản chất lưu trữ	Stateless (Tự chứa thông tin)	Stateful (Quản lý trạng thái trên Redis)
Nơi lưu trữ Server	Không lưu trên Server (RAM Client)	Redis Cache (<code>rt</code> , <code>rt_grace</code> , <code>rt_revoked</code>)
Thời hạn sống (TTL)	Ngắn hạn (1 giờ)	Dài hạn (24 giờ đến 30 ngày)
Vị trí phía Client	Header <code>Authorization: Bearer <at></code>	Cookie <code>HttpOnly: true</code> , <code>SameSite=Lax</code> , <code>Secure</code>
Chi phí thẩm định	Cực thấp (< 0.05ms trong RAM)	Trung bình (1 lần truy vấn Redis qua Pipeline)
Khả năng thu hồi	Hết hạn tự nhiên theo thời gian	Thu hồi tức thì bằng lệnh <code>DEL</code> trong Redis
Phòng vệ đặc thù	Thời gian sống ngắn hạn chế rủi ro	Token Rotation + Grace Period 30s + Reuse Detection